Islamic University of Technology

Lab 14: String, Structure

CSE 4108 : Structured Programming Lab

Section 1B

MIA, NFS, IAC, ASH

March 11, 2025

1 String: Instructions

This is a special lab. Complete all the tasks and submit them within time. Do not plagiarise (copy from others or the internet directly). Any kind of plagiarism, if caught, will be severely punished. There are two different sections of lab tasks. First one being implementation of your own string functions where as, the second section focuses on usage of string.

2 Lab Tasks: Custom String Manipulation

In this section of tasks, you are not permitted to use any string library functions (functions found in `<string.h>`). However, you can use `gets` and `puts` or other string scanning and printing functions. However, if you have designed your own function for a task, you can reuse it. For instance, if you have written your own `myStrLen` function to calculate string length and you need string length in `myStrNCat` function, you can use your `myStrLen`. For this section, submit only 1 C file containing all the functions and a main function properly testing all of them. Remember, for sensitive functions, test all possible different scenarios. Also, you cannot name any function with identifier beginning with small case 'str', it is a rule of C standard. And all the function declarations contain comments before them stating their parameters and return type.

2.1 String Comparison

Write your own function in C that mimics the behavior of strcmp function from string.h. That means it takes two string **variables** and/or string literals as input and lexicographically compares them. The function returns an integer, the value of which determines whether the first string is greater, equal to or smaller than the second string. Remember the two rules of lexicographical comparisons.

1. If first i characters matches but i + 1th character of first string is larger than the i + 1th character of second string, than first string is greater. Vice-versa for lesser.
2. If all the valid character matches but first string is smaller in length than the second string, then the first string is lesser. Vice-versa for greater.
3. (This rule is implicitly covered in the previous two, this is the otherwise condition) If and only if the lengths and all the characters match, then both strings are equal.

The declaration looks like below.

```
/*
Lexicographically compares two strings
Parameters:
    s1 [const char*]: First string
    s2 [const char*]: Second string
Return:
    [int]: Comparison result.
        If less than 0, then s2 is greater.
        If greater than 0, s1 is greater.
        If equal to 0, s1 and s2 are equal.
*/
int myStrCmp(const char *s1, const char *s2);
```

2.2 String Reverse

Write your own function in C that mimics the behavior of strrev function from string.h. That means it takes a string as input and reverses the string in place. For instance, if sentence strings holds the string "Hello World" after calling strrev on it, sentence will hold "dlroW olleH". The function returns a pointer to the input string to chain functions. The declaration of the function looks like below.

```
/*
Reverses a string in place
Parameters:
    s [char*]: Input string
Return:
    [char*]: Pointer to s
*/
char *myStrRev(char *s);
```

3 Lab Tasks: Using Strings to Solve Problems

In this section of tasks, you can use both your custom functions or the ones from the library. But we would appreciate if you are confident enough and can do these tasks using your own functions from the previous section.

3.1 Palindrome Done in Two

If you think a little bit carefully and do not consider efficiency for now, determining whether a word is palindrome or not is easy. You just compare whether the original world and its reversed version are equal or not. And there are functions for both of these tasks. So, determine whether a word is palindrome or not using string functions.

Design the input and output accordingly. Remember, you can use gets to easily scan words and puts to print them. You can consider that a word will never be more than 20 characters.

[Hint] *Remember, string reversing is done in place. So you should hold a copy of the original string first to compare the reverse with.*

4 Structure: Instructions

This is a special lab. Complete all the tasks and submit them within time. Do not plagiarise (copy from others or the internet directly). Any kind of plagiarism, if caught, will be severely punished. If you can make a multi-file implementation, with the calculation functions being in different header and source files and the **main function holding testing statements or routines**, it will be very much appreciated. The books contains one such example in the respective chapter. However, if you submit just one C files with all the logic, that is also fine. Keep the code clean. If you are using a single C file, name it as "L13_ID.c". In case of multi-file setup, name according to your convenience. Also, **there is no sub-task, the different stages of a single task is broken down into subsections**.

5 Lab Tasks: A Vector Three for Three

5.1 The Structure that Starts It All

A Vector3 or sometimes written as float3 or double3 is basically a structure containing 3 floating point values. A Vector3 can be used to represent the coordinates of a point in a 3 dimensional space. They can also be used to represent colors or rotations in Euler space. First, make a structure with tag _Vec3 and type definition of Vec3 with 3 double members.

```
Vec3
 |- x : double
 |- y : double
 |- z : double
```

You can assume that the x, y and z values are the 3 different coordinates in 3 dimensional space.

5.2 Functions that Manipulates

Once you have the structure at hand, implement the following functions that works with the Vec3 structure. The required formulae are given in section 6. Alternatively, you can Google a formulae if you need even more explanation. You might want to declare one or two extra functions like Radian to Degree conversion function to help you debug and verify codes that return values in Radian.

```
1  // Returns a Vec3 type data from passed values
2  Vec3 Vec3_Make(double x, double y, double z);
3
4  // Adds two Vec3s and returns the resultant Vec3
5  Vec3 Vec3_Add(const Vec3 *v1, const Vec3 *v2);
6
7  // Subtracts two Vec3s and returns the resultant Vec3
8  Vec3 Vec3_Sub(const Vec3 *v1, const Vec3 *v2);
9
10 // Returns the magnitude of a Vec3
11 double Vec3_Magnitude(const Vec3 *v);
12
13 // Returns the dot product of two Vec3s
14 double Vec3_Dot(const Vec3 *v1, const Vec3 *v2);
15
16 // Prints a Vec3
17 void Vec3_Print(const Vec3 *v);
```

Once you have implemented all the functions, in the main function, declare some Vec3s and run and test each of the function to test whether they are working properly or not.

5.3 Structures That Reside Inside Another

The lab will not be complete with nested structures. And unions or enums are not given since it was late to cover to them. Anyways, now make another structure with tag _Line and type definition Line. This structure will represent a line in a 3 dimensional space. The Line structure will contain two points or Vec3s, Start and End to represent the starting and ending coordinates respectively.

```
Line
 |- Start : Vec3
 |- End : Vec3
```

5.4 Some More Manipulative Ones

With the nested structure at your disposal, implement the following functions that work with the Line structure.

```
// Makes a line from input Vec3s
Line Line_Make(Vec3 start, Vec3 end);

// Prints the starting and ending Vec3 of a line without using pointer
void Line_RPrint(Line line);
```

Again, once you are done implementing these functions, test them out in the main function after the functions of Vec3. If you are done implementing all these functions and testing them, you are done. You can submit your task now.

6 Formulae

Let uppercase $V, A, B...$ represent Vector3s and lowercase $v, a, b...$ represent scalars. The members of a vector V_n will be written as x_n, y_n, z_n. The formulae are given in table 1.

Description	**Notation**	**Formula**
Vector Vector Addition	$V_1 + V_2$	$x_1 + x_2, y_1 + y_2, z_1 + z_2$
Vector Vector Subtraction	$V_1 - V_2$	$x_1 - x_2, y_1 - y_2, z_1 - z_2$
Vector Magnitude	$\|V\|$	$\sqrt{x^2 + y^2 + z^2}$
Vector Vector Dot	$V_1 \cdot V_2$	$x_1 * x_2 + y_1 * y_2 + z_1 * z_2$

Table 1: Formulae Table